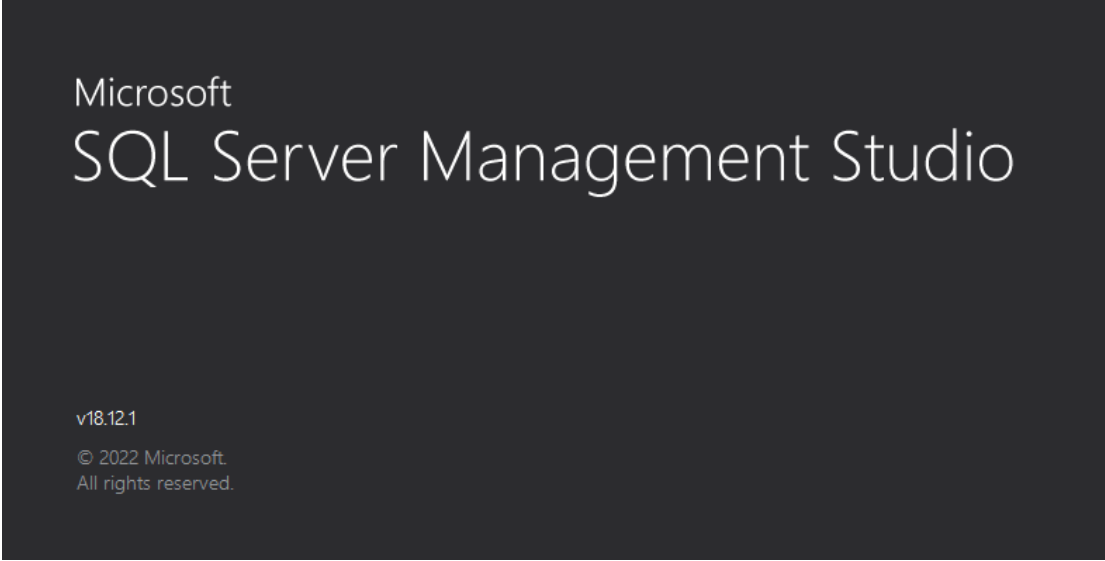


# Esquema interno – Ejemplo

Del DER al SQL Server



Microsoft  
SQL Server Management Studio

v18.12.1

© 2022 Microsoft.  
All rights reserved.

# Esquema interno – Ejemplo

## **Numéricos exactos - Enteros**

Bigint, numeric, bit, smallint, Int, tinyint, integer

## **Numéricos aproximados - Reales**

Float, decimal, smallmoney, money, real, Fecha y hora, date, datetimeoffset, datetime2, smalldatetime, datetime, time

## **Cadenas de caracteres**

Char(longitud), varchar(longitud), text

## **Cadenas de caracteres Unicode**

Nchar, nvarchar, ntext

## **Cadenas binarias**

Binario, varbinary, imagen

Para más info:

[https://www.w3schools.com/sql/sql\\_datatypes.asp](https://www.w3schools.com/sql/sql_datatypes.asp)

## **Otros tipos de datos**

Cursor, rowversion, hierarchyid, uniqueidentifier, sql\_variant, xml, Tipos de geometría espacial, Tipos de geografía espacial, table

# Teoría de Normalización

Antes del método de Codd, año 1970 (creador del modelo relacional), las bases de datos se almacenaban en un solo archivo.

- En un diseño de una BD. se obtienen unas tablas, pero no podemos estar seguros de que no presenten problemas:
  - Incapacidad de almacenar ciertos hechos.
  - Redundancias.
  - Ambigüedades.
  - Pérdida de información.
- Las reglas formales que forman la teoría de la normalización permiten detectar y corregir esos errores.
- Existen 6 FN, aunque lo normal es aplicar las 3 primeras FN.

# Teoría de Normalización

La normalización es el proceso mediante el cual se transforman datos complejos a un conjunto de estructuras de datos más pequeñas, que además de ser más simples y más estables, son más fáciles de mantener.

La normalización se utiliza para mejorar el esquema lógico, de modo que satisfaga ciertas restricciones que eviten la duplicidad de datos.

# Objetivos de la Normalización

- Minimizar la redundancia.
- Minimizar el mantenimiento de datos.
- Minimizar el impacto de futuros cambios (anomalías de actualización y anomalías de borrado) de datos, e ingreso de información (anomalías de inserción).

# Ventajas de la Normalización

- Evita anomalías en inserciones, modificaciones y borrados.
- Mejora la independencia de datos.
- No establece restricciones artificiales en la estructura de los datos.
- Facilidad de uso.
- Flexibilidad.
- Precisión.
- Seguridad. Facilidad de implementación.
- Independencia de datos.
- **Claridad.**
- Facilidad de gestión.
- Mínima redundancia.
- Máximo rendimiento de las aplicaciones.

# Teoría de Normalización

Existen 5 Formas Normales y una variante de la 4ta:

- Primera Forma Normal (1FN)
- Segunda Forma Normal (2FN)
- Tercera Forma Normal (3FN)
- Forma Normal de Boyce Codd (FNBC)
- Cuarta Forma Normal (4FN)
- Quinta Forma Normal (5FN)

# Teoría de Normalización

Tomaremos el siguiente ejemplo para ver las formas normales y reconocer el funcionamiento de cada una de ellas:

## FACTURACIÓN

***Codigo\_factura***

Codigo\_cliente

Nombre\_cliente

Direccion\_cliente

Localidad\_cliente

Fecha\_factura

Forma\_pago

Codigo\_articulo\_1

Descripcion\_1

Cantidad\_1

Importe\_1

Tipo\_IVA\_1

...

Codigo\_articulo\_N

Descripcion\_N

Cantidad\_N

Importe\_N

Tipo\_IVA\_N

# 1FN (Primera Forma Normal)

- Una base de datos esta en 1FN si:
  - Cada atributo de una tabla contiene un valor atómico (simple).

## FACTURA

|                         |
|-------------------------|
| ★ <u>Codigo_factura</u> |
| Codigo_cliente          |
| Nombre_cliente          |
| Direccion_cliente       |
| Localidad_cliente       |
| Fecha_factura           |
| Forma_pago              |

**Clave primaria**

## DETALLE\_FACTURA

|                          |
|--------------------------|
| ★ <u>Codigo_factura</u>  |
| ★ <u>Codigo_articulo</u> |
| Descripcion              |
| Cantidad                 |
| Importe                  |
| Tipo_IVA                 |

**Definición formal:** “ una relación está en 1FN, si cumple la propiedad de que sus dominios no tienen elementos que, a su vez, sean conjuntos.”

## 2FN (Segunda Forma Normal)

Una base de datos esta en 2FN si:

- Esta en 1FN.
- Cada atributo **no clave** depende de la clave completa y no de parte de ella.
  - Toda tabla con clave, formada por un solo atributo cumple con esta propiedad.

### FACTURA

Codigo\_factura  
Codigo\_cliente  
Nombre\_cliente  
Direccion\_cliente  
Localidad\_cliente  
Fecha\_factura  
Forma\_pago

### DETALLE\_FACTURA

Codigo\_factura  
Codigo\_articulo  
Cantidad  
Importe

### ARTICULO

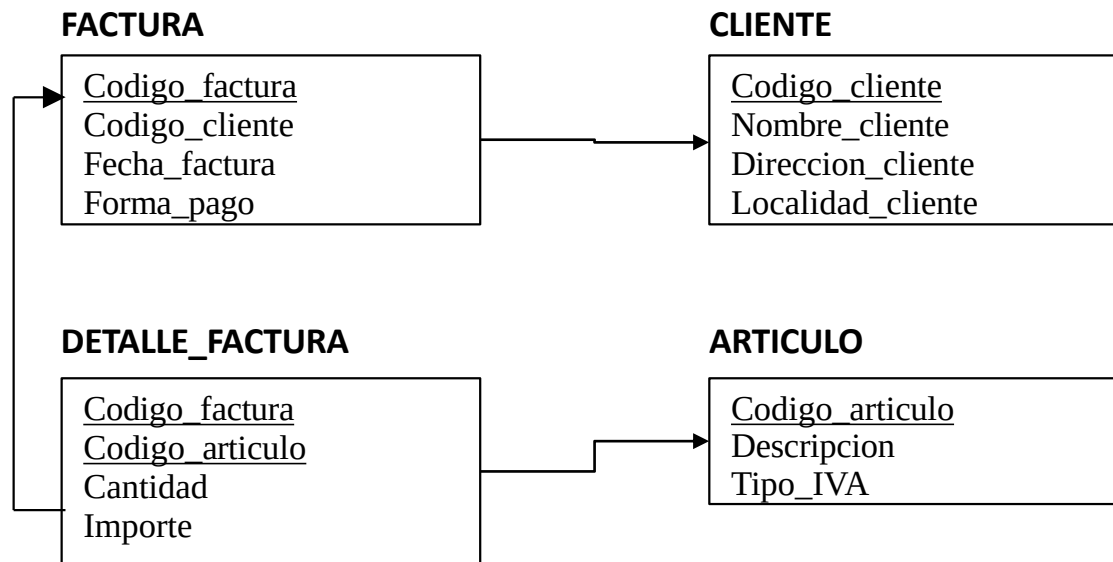
Codigo\_articulo  
Descripcion  
Tipo\_IVA

*La definición formal es: “se dice que una relación R está en 2FN, si además de estar en 1FN, cualquiera de sus atributos no-primarios tienen una dependencia funcional plena con cada una de las claves candidatas de R”.*

# 3FN (Tercera Forma Normal)

Una base de datos esta en 3FN si:

- Esta en 2FN.
- Todos los atributos que no son claves son independientes entre si.



***La definición formal es: “una relación R está en 3FN si esta en 2FN y además ninguno de sus campos no-primarios tiene dependencias transitivas respecto de las claves candidatas de R.”***

# Otras formas normales

## Investigar desde la teoría del aula virtual

- *Archivo: 05 - Teoria\_de\_Normalizacion.pdf*
- Forma Normal de Boyce Codd (FNBC)
- Cuarta Forma Normal (4FN)
- Quinta Forma Normal (5FN)

# Consideraciones Finales

Si se realiza un diseño conceptual y lógico:

- Aplicar las formas normales nos comprueba que es correcto.
- Ayuda a corregir los errores existentes.

Sin embargo, las FN implican descomponer tablas en otras más pequeñas.

- Problema de integridad de la base de datos.
- Disminución del rendimiento del sistema.

**Por tanto es necesario llegar a un punto intermedio, que garantice la “normalización de los datos” a un costo operativo aceptable.**

# Claves en una BD

Una clave en una base de datos es un atributo o conjunto de atributos que identifican a un registro.

## Clave Primaria

La clave primaria de una relación es un atributo o conjunto de atributos que identifican unívocamente a una tupla. Es decir, que todos los elementos del conjunto son únicos por cada registro.

| Cientes           |
|-------------------|
| * <b>Código</b>   |
| Nombre y Apellido |
| CUIT              |
| Domicilio         |

| Items de factura        |
|-------------------------|
| * <b>Codigo_factura</b> |
| * <b>Item_factura</b>   |
| Cantidad                |
| Precio                  |

# Claves en una BD

## Clave Candidata

La clave candidata es aquella columna que cumple todos los requisitos de una clave principal. En otras palabras, tiene potencial para ser una clave principal.

| Cientes           |
|-------------------|
| * <b>Codigo</b>   |
| Nombre y Apellido |
| CUIT              |
| Domicilio         |



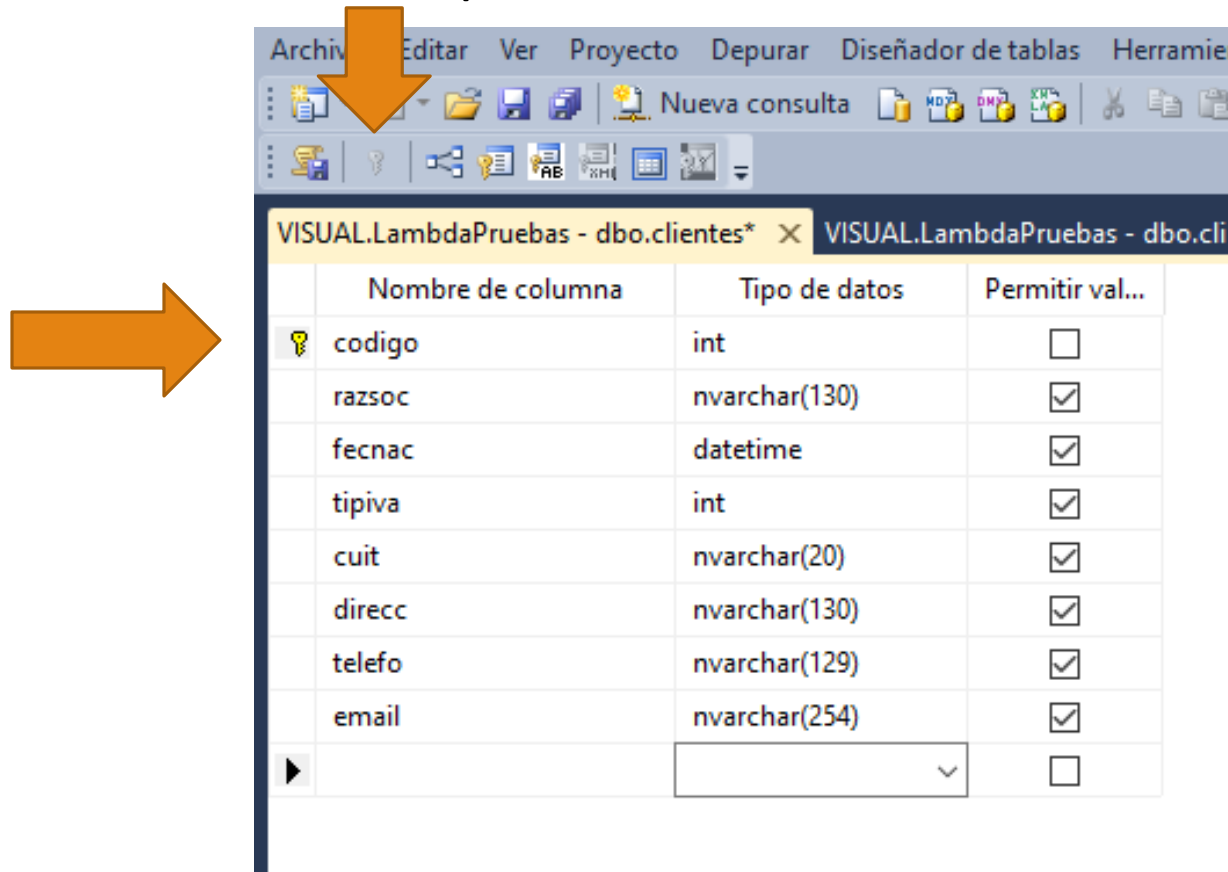
Tener en cuenta cada implementación. Puede que la CUIT cambie para un mismo cliente. En este caso dejaría de ser útil como clave primaria



En este ejemplo, CUIT podría reemplazar a **Codigo** como clave primaria ya que *podría* identificar al registro unívocamente.

# Claves en una BD

## Creación de una clave primaria en SQL Server



Visual Studio Enterprise Designer - Diseñador de tablas

Archivos Editar Ver Proyecto Depurar Diseñador de tablas Herramientas

Nueva consulta

VISUAL.LambdaPruebas - dbo.clientes\* X VISUAL.LambdaPruebas - dbo.cli

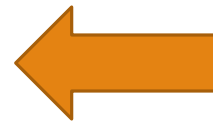
|   | Nombre de columna | Tipo de datos | Permitir val...                     |
|---|-------------------|---------------|-------------------------------------|
| 🔑 | codigo            | int           | <input type="checkbox"/>            |
|   | razsoc            | nvarchar(130) | <input checked="" type="checkbox"/> |
|   | fecnac            | datetime      | <input checked="" type="checkbox"/> |
|   | tipiva            | int           | <input checked="" type="checkbox"/> |
|   | cuit              | nvarchar(20)  | <input checked="" type="checkbox"/> |
|   | direcc            | nvarchar(130) | <input checked="" type="checkbox"/> |
|   | telefo            | nvarchar(129) | <input checked="" type="checkbox"/> |
|   | email             | nvarchar(254) | <input checked="" type="checkbox"/> |
| ▶ |                   |               | <input type="checkbox"/>            |

# Claves en una BD

Creación de una clave primaria a través del lenguaje de manipulación:

```
CREATE TABLE [usuario].[clientes] (  
    [codigo] [int] NOT NULL,  
    [razsoc] [nvarchar](130) NULL,  
    [fecnac] [datetime] NULL,  
    [tipiva] [int] NULL,  
    [cuit] [nvarchar](20) NULL,  
    [direcc] [nvarchar](130) NULL,  
    [telefono] [nvarchar](129) NULL,  
    [email] [nvarchar](254) NULL,  
    CONSTRAINT [PK_clientes] PRIMARY KEY CLUSTERED  
    (  
        [codigo] ASC  
    )  
)
```

Es el nombre de usuario de la tabla. Por defecto toma el usuario logeado



# Claves en una BD

Una clave primaria, como se mencionó anteriormente, debe identificar unívocamente a un registro, por lo que de ser necesario, esta puede estar compuesta por más de un registro.

```
CREATE TABLE [usuario].[items_factura] (  
    [codigo_factura] [int] NOT NULL,  
    [item_factura] [int] NOT NULL,  
    [cantidad] [float] NULL,  
    [codigo_articulo] [int] NOT NULL,  
    [precio_unicatio] [money] NULL,  
CONSTRAINT [PK_items_factura] PRIMARY KEY CLUSTERED  
(  
    [codigo_factura] ASC, [item_factura] ASC)  
)
```



# Claves en una BD

## Clave secundaria o índice secundario

Al igual que se estudió en archivos indexados, un índice es una estructura que mantiene ordenada según un criterio a la tabla.

De esta manera, pueden existir claves secundarias, únicas o no, que permitan al SGBD ordenar una tabla por otro criterio que no sea el establecido por la propia clave primaria.

En el ejemplo de la derecha, **Código** es clave primaria e identifica unívocamente a la factura, pero el subconjunto {Tipo\_comprobante, Letra, Sucursal, Número}, como superclave, también es única para la tabla.

### Cabecera\_Factura

\* **Código**

Tipo\_Comprobante

Letra

Sucursal

Número

Fecha\_Emision

Cod\_Cliente

Domicilio\_Entrega

# Claves en una BD

```
CREATE TABLE [dbo].[Cabecera_factura] (  
    [codigo] [int] NOT NULL,  
    [tipo_Comprobante] [char](2) not NULL,  
    [letra] [char](1) NULL,  
    [sucursal] [int] NULL,  
    [numero] [int] NULL,  
    [Fecha_Emission] [DateTime] NULL,  
    [Cod_cliente] [int] NOT NULL,  
    [Domicilio_Entrega] [nvarchar](254) NULL,  
  
    CONSTRAINT [PK_CabeceraFactura] PRIMARY KEY CLUSTERED  
    ( [codigo] ASC)  
)  
  
CREATE UNIQUE INDEX ix_numeroFactura  
    ON Cabecera_Factura (tipo_comprobante, letra,  
                        sucursal, numero)
```



## Cabecera\_Factura

**\* Codigo**

Tipo\_Comprobante

Letra

Sucursal

Número

Fecha\_Emission

Cod\_Cliente

Domicilio\_Entrega

# Claves en una BD

Por otro lado, cómo se mencionó anteriormente, un índice no necesariamente es único, sino que puede ayudarnos con la performance de una consulta.

Si el sistema cuenta con un listado tipo hoja de ruta en donde siempre se ordena por Domicilio de Entrega, entonces podremos definir un índice para que cree un árbol ordenado por este campo:

```
CREATE INDEX ix_Domicilio_Entrega  
ON Cabecera_Factura (Domicilio_Entrega)
```



## Cabecera\_Factura

**\* Código**

Tipo\_Comprobante

Letra

Sucursal

Número

Fecha\_Emision

Cod\_Cliente

Domicilio\_Entrega

# Claves en una BD

## Claves foráneas o ajenas

Una clave de este tipo es un atributo dentro de la tabla que apunta a un registro único en otra tabla.

En nuestro ejemplo, **Cod\_Cliente** hace referencia a la clave primaria de la tabla ***Clientes***. En este caso se puede definir una clave foránea que nos ayudará a mantener la consistencia entre los datos.

```
ALTER TABLE [Cabecera_factura]
ADD CONSTRAINT FK_Cod_Cliente FOREIGN KEY (Cod_cliente)
REFERENCES Clientes (Codigo)
ON DELETE CASCADE
ON UPDATE CASCADE
;
```



### Cabecera\_Factura

\* **Codigo**

Tipo\_Comprobante

Letra

Sucursal

Número

Fecha\_Emision

Cod\_Cliente

Domicilio\_Entrega

# Claves en una BD – Claves foráneas

```
ALTER TABLE [Cabecera_factura]
ADD CONSTRAINT FK_Cod_Cliente FOREIGN KEY (Cod_cliente)
REFERENCES Clientes (Codigo)
ON DELETE CASCADE
ON UPDATE CASCADE
;
```

Cómo se muestra en la sintaxis SQL, existen dos atributos importantes (Restricciones) en la creación de una clave foránea que indican qué debe hacer el SGBD ante un caso de actualización o eliminación del atributo: ***ON DELETE***, ***ON UPDATE***.

Especificando el atributo **NO ACTION**, se indica que si intentamos eliminar o actualizar un valor de la clave primaria de la tabla referenciada (Clientes) que tengan referencia en la tabla principal (En nuestro ejemplo Cabecera\_Factura), se genere un error y la acción no se realice; es la opción predeterminada.

Especificando **CASCADE**, se indica que si eliminamos o actualizamos un valor de la clave primaria en la tabla referenciada (Clientes), los registros coincidentes en la tabla principal (Cabecera\_Factura), también se eliminan o modifiquen; es decir, si eliminamos o modificamos un valor de campo definido con una restricción "primary key" o "unique", dicho cambio se extiende al valor de clave externa de la otra tabla (integridad referencial en cascada).